

Git y Fork — Guía para principiantes

Todo lo que necesitas entender antes de usar Fork, explicado desde cero

1. ¿Qué es Git y qué es Fork?

Git es un sistema de control de versiones: guarda un historial de todos los cambios que haces en tus archivos, permitiéndote volver atrás, comparar versiones o trabajar en paralelo con otras personas sin perder trabajo.

Fork es un programa con interfaz gráfica que te permite usar Git sin escribir comandos en la terminal. Es decir, Git es el "motor" y Fork es el "volante y salpicadero" que lo hacen manejable visualmente.

Idea clave: Git no vive "en internet". Vive en tu ordenador, dentro de una carpeta especial llamada `.git` que se crea dentro de tu proyecto (repositorio). Solo cuando tú lo decides, envías esos cambios a un servidor remoto (como GitHub o GitLab).

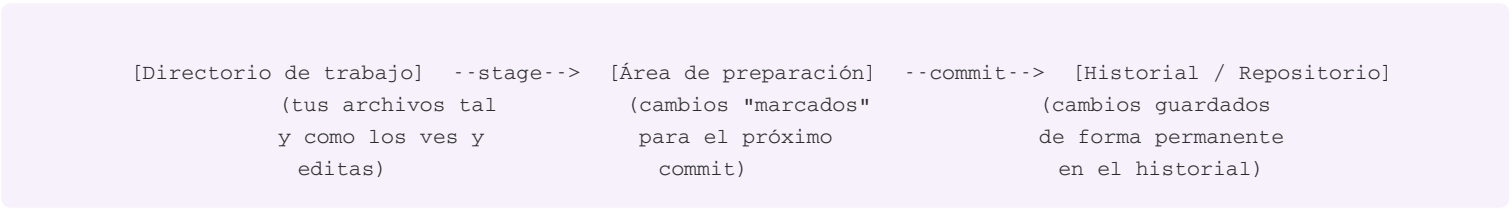
2. Conceptos esenciales (el vocabulario de Git)

Repositorio

Es la carpeta de tu proyecto que Git está vigilando. Dentro de ella, Git registra todos los cambios de todos los archivos a lo largo del tiempo.

Las tres "zonas" de Git

Para entender Git es fundamental visualizar que un archivo puede estar en tres estados o "zonas" diferentes:



Zona	Qué significa
Directorio de trabajo (Working Directory)	Los archivos tal cual están en tu carpeta ahora mismo. Aquí es donde editas, borras o creas archivos con normalidad.
Área de preparación (Staging Area / Index)	Una "zona intermedia" donde colocas los cambios que quieres incluir en el próximo commit. Puedes tener 5 archivos modificados pero decidir subir solo 2 de ellos.
Repositorio / Historial	Donde quedan guardados de forma permanente los commits ya confirmados. Es el "libro de historia" del proyecto.

Stage / Unstage (preparar / despreparar cambios)

- Stage** (hacer "staging" o "añadir al índice"): marcar un archivo o unas líneas concretas como "listas para ser guardadas" en el próximo commit. Es como decir: "esto sí quiero que forme parte del siguiente guardado".
- Unstage**: lo contrario. Quitar un archivo o líneas del área de preparación, sin perder el cambio (sigue estando en tu directorio de trabajo, simplemente ya no está "marcado" para el próximo commit).

Por qué es útil: te permite dividir tu trabajo en commits pequeños y ordenados. Por ejemplo, si has corregido un error y además añadido una función nueva, puedes hacer *stage* solo del error, confirmarlo (commit), y luego hacer *stage* de la función nueva por separado.

Commit

Un **commit** es una "fotografía" guardada de forma permanente de los cambios que están en el área de preparación, junto con un mensaje que describe qué se ha hecho. Cada commit tiene un identificador único y queda registrado en el historial para siempre (salvo que se manipule expresamente).

Piensa en cada commit como un punto de guardado en un videojuego: puedes volver a él cuando quieras.

Push y Pull

Acción	Qué hace
Push	Envía tus commits guardados localmente hacia el repositorio remoto (por ejemplo, GitHub), para que otras personas los vean.
Pull	Descarga los commits nuevos del repositorio remoto y los combina (fusiona) con tu copia local.
Fetch	Descarga la información del remoto (qué commits nuevos hay) pero <i>sin</i> mezclarlos todavía con tu trabajo. Es una forma de "mirar antes de tocar".

Stash

El **stash** es como un "cajón temporal": guarda tus cambios sin confirmar (sin hacer commit) para dejar el directorio de trabajo limpio, por ejemplo si necesitas cambiar de rama urgentemente sin perder lo que estabas haciendo. Más tarde puedes recuperarlos ("aplicar el stash") justo donde los dejaste.

Ramas (branches)

Una **rama** es una línea de desarrollo independiente. La rama principal suele llamarse `main` o `master`. Crear una rama nueva te permite probar cosas o desarrollar una función sin afectar al código principal hasta que estés listo para unirlo (merge).

Merge y conflictos

Merge es el proceso de unir los cambios de una rama con otra. Si dos ramas han modificado la misma línea de un mismo archivo de forma distinta, Git no puede decidir por ti: aparece un **conflicto de fusión**, y tienes que decidir manualmente qué versión conservar (o combinar ambas).

Log de commits (historial)

El **log** es la lista cronológica de todos los commits realizados en el repositorio. En Fork, este historial se muestra como un **grafo visual**: cada punto es un commit, y las líneas que los conectan muestran cómo se han creado y fusionado las ramas a lo largo del tiempo.

De cada commit del log normalmente puedes ver:

- El mensaje descriptivo del commit
- Quién lo hizo (autor) y cuándo
- Un identificador único (hash), por ejemplo `a1b2c3d`
- Qué archivos cambiaron y qué se modificó exactamente en cada uno (diff)

Tipos de cambios en un archivo

Cuando Git detecta que algo ha cambiado en tu proyecto, lo clasifica en una de estas categorías. Fork suele mostrarlas con letras o colores distintos en el panel de cambios:

Tipo de cambio	Letra habitual	Significado
Added (Añadido)	A	Es un archivo nuevo que no existía antes en el repositorio.
Modified (Modificado)	M	Un archivo ya existente ha sido editado (se han cambiado líneas dentro de él).
Deleted (Eliminado)	D	Un archivo que existía ha sido borrado.
Renamed (Renombrado)	R	Un archivo ha cambiado de nombre o ubicación (Git suele detectarlo automáticamente si el contenido es muy similar).
Untracked (Sin seguimiento)	U / ?	Un archivo nuevo que Git todavía no está vigilando; no se incluirá en ningún commit hasta que lo añadas (stage) por primera vez.

Diff (diferencia)

El **diff** es la comparación visual entre dos versiones de un archivo. Normalmente se muestra con líneas en **rojo** (lo que se ha eliminado) y en **verde** (lo que se ha añadido). Fork ofrece una vista "lado a lado" muy clara para ver estas diferencias.

3. La interfaz de Fork: qué es cada cosa

Zona de la pantalla	Qué encontrarás
Barra lateral izquierda	Ramas locales, ramas remotas, etiquetas (tags) y stashes guardados.
Lista de commits (centro)	El historial (log) en forma de grafo visual, con cada commit y sus ramificaciones.
Working Directory Changes	Lista de archivos modificados, nuevos o eliminados que todavía no se han confirmado (commit).
Panel de diff	Comparación lado a lado del contenido antes/después de un archivo seleccionado.
Barra superior	Botones de acción rápida: Fetch, Pull, Push, Branch, Stash, etc.

4. Primeros pasos en Fork (flujo básico)

Paso 1 — Obtener un repositorio

- **Clonar uno existente:** menú `File > Clone` , pega la URL del repositorio remoto y elige una carpeta en tu ordenador.
- **Abrir uno que ya tienes:** `File > Open` y selecciona la carpeta.
- **Crear uno nuevo desde cero:** `File > New` .

Paso 2 — Editar archivos con normalidad

Modifica, crea o borra archivos en la carpeta del proyecto como harías normalmente (con tu editor de código, por ejemplo). Git no hace nada todavía: solo "observa".

Paso 3 — Revisar los cambios

Abre Fork y mira el panel **Working Directory Changes**. Ahí verás la lista de archivos que han cambiado, con su tipo de cambio (añadido, modificado, eliminado).

Paso 4 — Hacer stage de los cambios

Selecciona los archivos (o incluso líneas concretas dentro de un archivo) que quieras incluir en el próximo commit y márcalos como "stage". Los que no marques se quedarán pendientes para más adelante.

Paso 5 — Hacer commit

Escribe un mensaje breve y descriptivo (por ejemplo: "Corrige error en el formulario de contacto") y pulsa **Commit**. Esto guarda permanentemente esos cambios en tu historial local.

Paso 6 — Sincronizar con el remoto

- Pulsa **Push** para subir tus commits al servidor remoto.
- Pulsa **Pull** para bajar los commits nuevos que otras personas hayan subido.

Consejo para empezar: antes de hacer push, es buena práctica hacer pull primero, para asegurarte de que tienes la versión más reciente y evitar conflictos innecesarios.

Paso 7 — Trabajar con ramas (cuando lo necesites)

- **Crear una rama:** clic derecho sobre un commit o rama → *Create branch*.
- **Cambiar de rama:** doble clic sobre la rama en la barra lateral.
- **Unir ramas (merge):** sitúate en la rama de destino → clic derecho sobre la rama de origen → *Merge into current branch*.

Paso 8 — Guardar cambios temporalmente (stash)

Si necesitas cambiar de rama pero no quieres hacer commit todavía, usa **Stash** para guardar tus cambios a un lado. Podrás recuperarlos después con "Apply" o "Pop".

Paso 9 — Si aparece un conflicto

Fork señalará visualmente el archivo en conflicto y abrirá una herramienta que te muestra ambas versiones del cambio, para que elijas cuál conservar (o combines ambas manualmente).

5. Resumen visual de comandos y su equivalente en Fork

Concepto	Qué hace	En Fork
Stage	Marca un cambio para el próximo commit	Clic en el archivo/línea → "Stage"
Unstage	Quita un cambio del área de preparación	Clic en el archivo/línea → "Unstage"
Commit	Guarda permanentemente los cambios en stage	Escribir mensaje → botón "Commit"
Push	Sube commits locales al remoto	Botón "Push" (barra superior)
Pull	Baja y fusiona cambios del remoto	Botón "Pull" (barra superior)
Fetch	Consulta cambios remotos sin fusionarlos	Botón "Fetch" (barra superior)
Stash	Guarda cambios sin confirmar temporalmente	Botón "Stash"
Log	Ver historial de commits	Lista/grafa central de commits
Diff	Ver diferencias línea a línea	Panel de diff lado a lado
Branch	Crear/cambiar de rama	Barra lateral izquierda
Merge	Unir ramas	Clic derecho sobre la rama

6. Otras funciones útiles que ofrece Fork

- **Rebase interactivo:** reordenar, editar o unir (squash) varios commits de forma visual.
- **Blame:** ver quién modificó por última vez cada línea concreta de un archivo.
- **History:** buscar todos los commits en los que se modificó un archivo o carpeta específica.
- **Reflog:** recuperar commits que parecían "perdidos" (por ejemplo, tras deshacer algo por error).
- **Diffs de imágenes:** comparar visualmente archivos gráficos, no solo texto.
- **Notificaciones de GitHub** integradas directamente en la aplicación.
- Compatibilidad con **Git-flow**, **Git LFS** y **GPG** (firmas de commits).

7. Glosario rápido

Término	Significado en una frase
Repositorio	Proyecto vigilado por Git.
Working directory	Tus archivos tal como están ahora mismo.
Staging area	Zona intermedia de cambios "marcados" para el próximo commit.
Commit	Guardado permanente de cambios con un mensaje descriptivo.
Rama (branch)	Línea de desarrollo independiente.
Remoto (remote)	Copia del repositorio alojada en un servidor (ej. GitHub).
Merge	Unir los cambios de dos ramas.
Conflicto	Cuando Git no puede decidir automáticamente qué versión de un cambio conservar.
Stash	Cambios guardados temporalmente, sin hacer commit.
Log	Historial completo de commits.
Diff	Comparación visual entre dos versiones de un archivo.
Hash	Identificador único de cada commit.

